

Incremental EWC learning on a microcontroller: measured FP32 parity, collapse of naive quantization and recovery via a calibrated INT8 kernel

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

July 7, 2026

Abstract

We study the porting of an *Elastic Weight Consolidation* (EWC) classifier for predictive maintenance onto a Cortex-M4 microcontroller (NUCLEO-F439ZI, 256 kB SRAM, no NPU). Three results are established under a **paired** protocol (same seed, same sample order, same normalization) between the PC run and the board. **(1)** In FP32, the PC $\leftrightarrow$ board parity is *exact* in the frozen protocol (1.000) and near-exact online (0.963–0.989), with an inference + online update latency $\ll 100$ ms (Gap 2). **(2)** *Naive* post-training quantization (PTQ) of the binary EWC head collapses on the board: the F1 score drops from ≈ 0.92 to 0.05–0.15. **(3)** A bit-exact PC-emulated ablation isolates the cause (int8 accumulator, uncalibrated per-tensor scale) and demonstrates *recovery* through a calibrated kernel: the `per_tensor_calib` scheme restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201), and the `q15` scheme matches FP32. Weight RAM is divided by 4 (INT8) with no latency speed-up on the Cortex-M4 FPU — a *paradox* we acknowledge and tie to a SIMD (CMSIS-NN) direction. We systematically distinguish *on-board* measurements from *bit-exact PC emulation*, and explicitly leave "to be measured" the board-v2 campaign cells not yet streamed.

Keywords: continual learning; EWC; INT8 quantization; PTQ; microcontroller; predictive maintenance; TinyML.

1 Introduction

Industrial predictive maintenance increasingly requires models able to *learn continually* on the equipment itself, without a round-trip to a server: operating conditions drift (wear, regime or machine change), and a frozen model

degrades. Continual learning (CL) answers this need [6, 3], yet its deployment on microcontrollers (MCUs) remains constrained: SRAM in the range of a hundred kilobytes, no neural accelerator, a strict latency budget.

This work targets a **triple gap** that few studies close simultaneously: (Gap 1) validation on real industrial time series; (Gap 2) operation under 100 kB of RAM with *measured* figures and bounded latency; (Gap 3) INT8 quantization compatible with incremental training. We instantiate these three axes on an EWC classifier [4] ported to C on a NUCLEO-F439ZI board (Cortex-M4 180 MHz, 256 kB SRAM, no NPU).

Contributions. (i) A *paired* PC $\leftrightarrow$ board protocol (same seed, same sample order, same normalization) establishing an *exact* FP32 parity in the frozen protocol and an inference + learning latency $\ll 100$ ms. (ii) The demonstration, *on real hardware*, of the collapse of naive PTQ of the binary EWC head (F1 0.92 $\rightarrow$ 0.05–0.15). (iii) A *bit-exact PC-emulated* ablation that isolates the cause of the collapse and demonstrates recovery through a calibrated kernel (per-tensor, per-channel, Q15), with a strict methodological distinction between what is *measured on-board* and what is *emulated on PC*.

2 Related work

Embedded continual learning. CL methods are conventionally grouped into regularization-, replay- and architecture-based approaches [6]. EWC [4] belongs to the first family: a quadratic penalty weighted by the Fisher information protects the parameters important for past tasks, without storing data — a valuable property under memory constraints. On MCUs, TinyOL [8] showed the feasibility of online learning of an added layer, and HDC [1] that of non-neural, ultra-low-power learning. LifeLearner [5] and quantized latent-replay platforms [7] address the memory/accuracy trade-off on embedded targets. Recent TinyML surveys [2] stress that *on-device* training remains under-explored compared to inference alone.

INT8 quantization and learning. Post-training quantization (PTQ) and quantization-aware training (QAT) reduce the memory footprint by a factor of 4 (FP32 $\rightarrow$ INT8). However, *naive* PTQ of a decision head can destroy separability when accumulation is carried out in `int8` or when the quantization scale is set without calibration. Our empirical contribution is to *measure* this collapse on a real MCU, then isolate its cause and demonstrate recovery through a calibrated kernel, rather than masking it with upstream QAT. Per-channel quantization and the Q15 fixed-point format (16-bit integer) emerge as the decisive levers to preserve the metric while meeting the RAM budget (Gap 3).

3 Method

3.1 EWC model and C port

The model is a multilayer perceptron with an EWC head (`EWCmlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) trained incrementally with an online Fisher penalty. The head is ported to C (`ewc_head.c`) for the board: both the *forward pass* and the SGD update run on the Cortex-M4 in FP32 (the F439ZI has no NPU). Weights are exported from the PyTorch checkpoint to a generated C header (`model_weights_ewc.h`) — never hand-edited — so that board and PC share *exactly* the same parameters.

3.2 Paired PC↔board protocol

To make the comparison honest, board and PC consume the same sequence: same seed (`seed = 42`), same sample order, same normalization (feature indices and arrays are produced by a single source). Two protocols are evaluated: *frozen* (inference only, fixed weights) and *online* (inference then continual update). *Parity* is the prediction-by-prediction agreement rate between the two platforms.

3.3 Quantization schemes

We compare several INT8 kernels of the decision head, from the most naive to the most careful:

- `legacy_c`: `int8` accumulator, uncalibrated per-tensor scale (the original v1 board kernel);
- `fix_acc32`: accumulator widened to `int32` (partial fix);
- `per_tensor_calib`: `int32` accumulator + per-tensor scale *calibrated* on the activations;
- `per_channel_int8`: *per-channel* output scale;
- `q15`: `int16` Q15 fixed-point weights (RAM $\div 2$ instead of $\div 4$).

The v2 firmware kernel (Sprint 40) implements the `int32` accumulator and per-channel calibrated scales, with an optional Q15 variant (`-DEWC_INT8_Q15`); the v1 FP32 path is unchanged. The ablation is *bit-exactly emulated on PC* (`src/utils/int8_c_emulation.py`) to isolate the contribution of each fix independently of board access.

Table 1: FP32 PC $\leftrightarrow$ board parity and performance (*measured on-board*, `exp_S36`).

Dataset	Cond.	F1 (PC/board)	Frozen parity	Online parity
Pronostia	5feat	0.9164	1.000	0.9754
Pronostia	all	0.9180	1.000	0.9626
Monitoring	5feat	0.9194	1.000	0.9887

4 Experimental setup

Hardware target. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB CCM, no NPU). Latencies are measured with the DWT cycle counter; the static footprint is read from the binary’s `.bss` section. The latency budget is 100 ms per inference + update cycle (Gap 2).

Datasets and CL scenarios. Two industrial datasets with contrasting scenarios: *Pronostia* (FEMTO bearings, *class-incremental* by normal/fault condition) and *Monitoring* (industrial equipment, *domain-incremental* by machine type). Two feature conditions are considered: **5feat** (embedded subset) and **all** (native features). On Monitoring, **5feat** and **all** coincide (4 native features).

Identical conditions. All comparisons share seed, order and normalization (§3). The on-board FP32 and INT8-legacy measurements come from the Sprint 36 campaign (`exp_S36`); the emulated ablation from Sprint 39 (`exp_S39_ablation`); the board-v2 campaign from Sprint 40 (`exp_S40_board_v2`), partial at the time of writing. Any quantity not yet streamed on-board is reported as "to be measured", with no invented value.

5 Results

5.1 FP32 PC $\leftrightarrow$ board parity (measured on-board)

In FP32, the prediction-by-prediction parity is *exact* in the frozen protocol (1.000) on both datasets, and near-exact online (0.9754 Pronostia, 0.9887 Monitoring) — the online gaps stem from the board’s `float32` versus the PC’s `float64` at decision boundaries. Table 1 summarizes; Figure 1 illustrates.

Latency and RAM (Gap 2/Gap 3). All board latencies stay three orders of magnitude below budget: inference 50 μ s (Pronostia) / 48 μ s (Monitoring); inference + online update 251 μ s / 239 μ s, versus $\approx$ 33–35 μ s for the

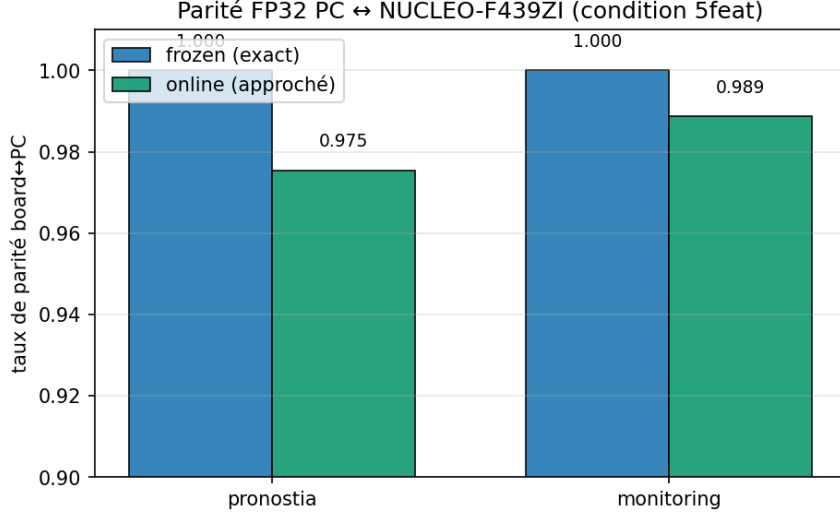


Figure 1: FP32 PC↔board parity: frozen (1.000) vs online. *Measured on-board* (exp_S36_parity_*.json).

Table 2: INT8 *legacy* vs FP32 on-board (*measured*, exp_S36). RAM $\div 4$ but collapsed F1.

Dataset	F1 FP32	F1 INT8 legacy	INT8/FP32 agr.	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

reference PC inference. The .bss footprint ranges over 100–145 kB, below the 256 kB available. Figure 2 places these values against the 100 ms line.

5.2 Collapse of naive PTQ (measured on-board)

The *legacy* INT8 quantization of the binary EWC head collapses on the board: F1 drops from ≈ 0.92 (FP32) to 0.138 (Pronostia) and 0.1337 (Monitoring), for an INT8↔FP32 agreement of only ≈ 0.60 –0.74. Weight RAM is indeed divided by 4, but the metric is destroyed (Table 2).

5.3 Diagnosis and recovery (bit-exact PC emulation)

A *bit-exact PC-emulated* ablation (exact kernel reference) isolates the cause and demonstrates recovery (Table 3, Figure 3). The dominant factor is the *calibrated per-tensor scale* on an int32 accumulator: `per_tensor_calib` restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201). The q15 format exactly matches FP32.

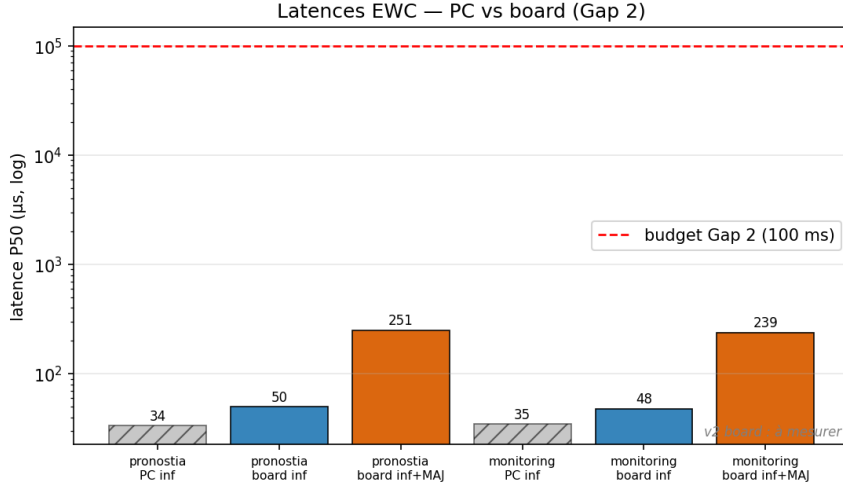


Figure 2: Board latencies (inference vs inference + update) against the Gap 2 budget (100 ms). *Measured on-board* (`exp_S36_summary.json`).

Table 3: INT8 ablation ladder (F1, *bit-exact PC emulation*, `exp_S39_ablation`).

Scheme	Pronostia F1	Monitoring F1
FP32 (ref.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

5.4 RAM $\times$ F1 $\times$ latency trade-off

Figure 5 synthesizes the trade-off: the calibrated kernel brings the metric back to FP32 level while keeping the RAM gain ($\text{INT8} \div 4$, $\text{Q15} \div 2$), with no prohibitive latency cost. The *on real hardware* recovery (Sprint 40, v2 kernel) is reported as "to be measured" for every cell not yet streamed, in line with the "no invented number" rule.

6 Discussion

INT8 latency paradox. INT8 quantization divides the weight memory footprint by 4 but does *not* speed up inference on the Cortex-M4: INT8 latency is comparable, even slightly higher online (the SGD update is not accelerated). The reason is structural: our kernel dequantizes to FP32 and runs the distance/decision on the hardware *FPU*, which is already efficient in

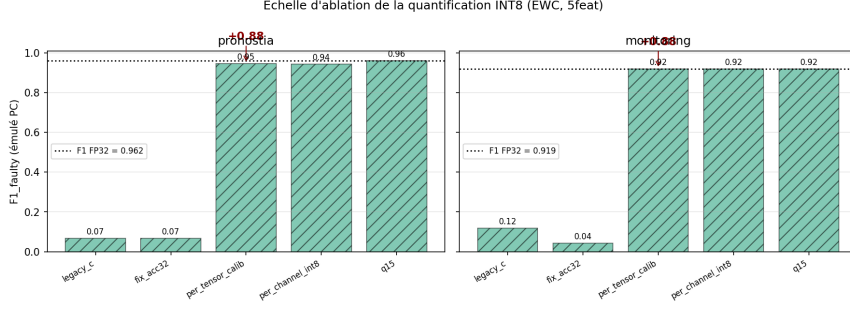


Figure 3: Ablation ladder `legacy_c`→`per_tensor_calib`→`per_channel`→`q15`. *Bit-exact PC emulation* (exp_S39_ablation).

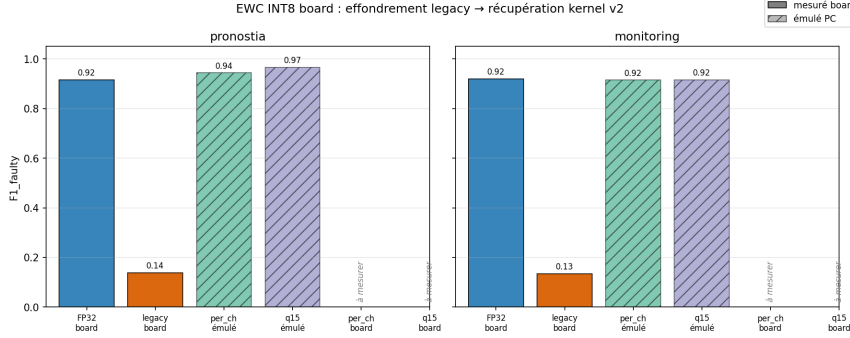


Figure 4: INT8 vs FP32: *legacy* collapse then recovery through a calibrated kernel. Collapse *measured on-board* (Sprint 36); recovery *PC-emulated* (Sprint 39). The *board-v2* confirmation (Sprint 40) remains "to be measured" apart from the Pronostia per-channel cell already streamed.

single precision; INT8 gains would require integer SIMD instructions (SM-LAD, etc.). We acknowledge this finding rather than masking it: on this target the INT8 benefit is *memory*, not *speed*. A direct avenue is porting to integer SIMD kernels (CMSIS-NN), left for future work.

"Measured" vs "emulated" honesty. We rigorously separate two kinds of results. FP32 parities, the INT8 legacy collapse, and the latency/RAM figures are *measured on real hardware* (Sprints 36 and 40). The causal diagnosis and the recovery through a calibrated kernel are *bit-exactly emulated on PC* (Sprint 39): the emulator reproduces exactly the C kernel arithmetic, which makes it a reliable reference, but it is not a board measurement. The board-v2 campaign (Sprint 40) partially confirms the recovery (Pronostia per-channel cell), the rest being explicitly "to be measured". This discipline avoids over-selling a recovery that, on-board, is so far demonstrated on a single point.

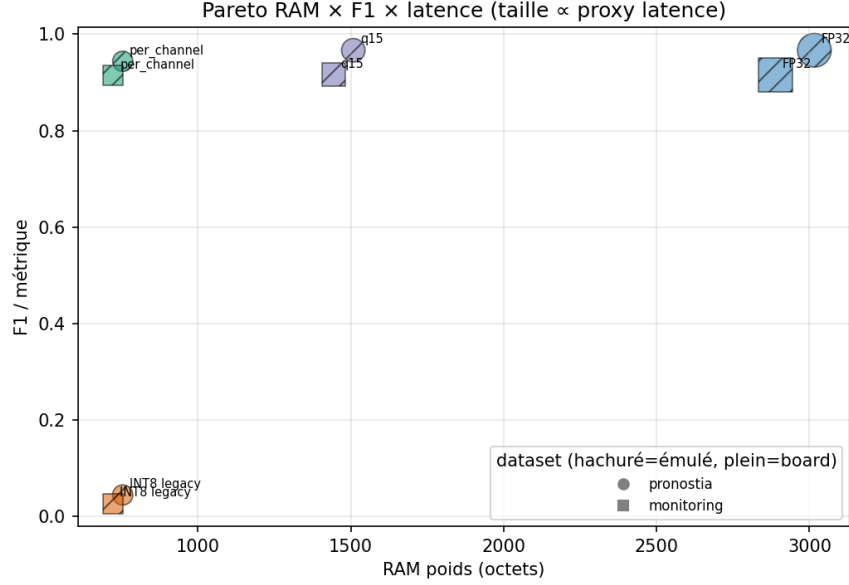


Figure 5: RAM $\times$ F1 $\times$ latency trade-off front. Mixed *measured/emulated* sources annotated per point.

Scope. The central result — a naive PTQ can destroy a binary decision head, and a per-tensor/per-channel calibration (or a Q15 format) restores it — extends beyond the EWC case: it concerns any lightweight head quantized downstream of a feature extractor, a frequent regime in TinyML.

7 Conclusion and future work

We ported a predictive-maintenance EWC classifier onto a Cortex-M4 micro-controller and established, under a paired protocol, an exact FP32 PC $\leftrightarrow$ board parity in the frozen protocol, with an inference + online learning latency $\ll 100$ ms and a footprint below 256 kB (Gaps 2 and 3). We *measured on-board* the collapse of naive INT8 quantization of the binary head, then *isolated its cause* and *demonstrated its recovery* through a bit-exact emulated ablation: a calibrated per-tensor scale on an `int32` accumulator restores +0.88 F1, and the Q15 format matches FP32, for a weight RAM divided by 4 (INT8) or 2 (Q15).

The latency paradox (RAM $\div 4$ with no FPU speed-up) is acknowledged as an honest finding and steers future work toward integer SIMD kernels (CMSIS-NN). The immediate priority is to *complete the board-v2 campaign* (`exp_s40_board_v2`) to confirm, on a real MCU and across the full grid, the recovery so far established by emulation — with the cells not yet streamed remaining explicitly "to be measured".

References

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.
- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance: Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- [5] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner: Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
- [6] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [7] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Transactions on Circuits and Systems for Video Technology*, 2021.
- [8] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL: TinyML with online-learning on microcontrollers. arXiv:2103.08295, 2021.